DESIGN AND ANALYSIS OF ALGORITHMS

DAA Viva Revision Guide | Easy definitions | Algorithms | Formulas | Time & Space Complexity

Prepared from the five uploaded DAA assignment PDFs. Content is reorganized topic-wise for viva revision rather than following the PDF page order. Formulas are typed in plain text to avoid unreadable symbols.

Included practicals: Fibonacci (Assignment 1), Huffman Encoding (Assignment 2), 0/1 Knapsack (Assignment 3), N-Queens using Backtracking (uploaded Assignment 4 file, whose title page says Assignment No. 5), and Deterministic/Randomized Quick Sort (Assignment 5).

1. Essential DAA fundamentals

Term	Short and easy definition
Algorithm	A finite, step-by-step method to solve a problem.
Data structure	A way to organize and store data so it can be used efficiently.
Time complexity	How the number of operations grows as input size increases.
Space complexity	How much extra memory an algorithm needs as input size increases.
Big-O, O(...)	An upper-bound description of growth; commonly used for worst-case analysis.
Theta, Θ(...)	A tight bound: the growth rate is bounded both above and below.
Omega, Ω(...)	A lower-bound description of growth.
Recursion	A function solves a problem by calling itself on smaller inputs.
Iteration	Repeating steps using loops.
Auxiliary space	Extra memory used by the algorithm, excluding the input itself.
Best case	The least work for an input of a given size.
Average case	Expected work over a specified distribution of inputs.
Worst case	The maximum work for any input of a given size.

Common complexity growth (fast to slow)

```
O(1) < O(log n) < O(n) < O(n log n) < O(n^2) < O(2^n) < O(n!)
```

Viva tip: Always state what n means. For Huffman, n can mean the number of distinct characters; for Knapsack, n is the number of items and W is the capacity; for N-Queens, n is the board size.

2. Practical 1 — Fibonacci Series

Definition and recurrence

Fibonacci series: A sequence where each term is the sum of the previous two terms. It starts with 0 and 1.

```
0, 1, 1, 2, 3, 5, 8, 13, 21, ...
F(0) = 0; F(1) = 1; F(n) = F(n-1) + F(n-2), for n >= 2
```

Golden ratio: The ratio of two consecutive Fibonacci numbers approaches approximately 1.618 as the terms become large.

Algorithms and complexity

Method	How it works	Time	Auxiliary space
Naive recursion	Calls F(n-1) and F(n-2) repeatedly; same values are recalculated.	O(2^n) (exponential upper bound; more tightly Θ(phi^n))	O(n) call stack
Dynamic programming / tabulation	Stores previously calculated Fibonacci values in an array and builds from small to large.	O(n)	O(n)
Space-optimized iteration	Keeps only the previous two values and updates them in a loop.	O(n)	O(1)
Memoization	Recursive solution plus a cache; each F(k) is calculated once.	O(n)	O(n) cache + stack
Fast doubling	Computes Fibonacci values using identities that halve the input at each step.	O(log n)	O(log n) recursive stack; O(1) iterative

Iterative idea

```
a = 0; b = 1 Repeat n terms: print a c = a + b a = b b = c
```

Recursive idea

```
FIB(n): if n <= 1: return n return FIB(n-1) + FIB(n-2)
```

Why iteration is faster than simple recursion

• Naive recursion solves the same subproblems many times.

• Iteration calculates each next term once, so its time is linear.

• Keeping only two previous terms reduces extra memory to constant space.

Other viva points

• Fibonacci search: A search technique for a sorted array that divides the search range using Fibonacci numbers. The supplied practical asks its definition but does not provide its implementation or complexity analysis.

• Applications mentioned in the PDF: number patterns in nature, coding and distributed systems, analysis of Euclid's algorithm, science, cryptography and Fibonacci retracement in financial technical analysis.

• The uploaded PDF lists matrix-power and direct-formula approaches too, but does not explain their implementation in detail. A common fast-doubling approach is O(log n); direct closed-form calculation has practical numeric-precision limitations.

3. Practical 2 — Huffman Encoding (Greedy)

Definitions

Term	Easy meaning
Greedy method	At each step, choose the best-looking available option at that moment.
Huffman coding	A lossless compression method that gives shorter codes to frequent characters and longer codes to rare characters.
Prefix code	A code in which no character's complete code is the starting part of another character's code.
Huffman tree	A binary tree whose leaves represent characters and whose paths give their codes.
Min-heap / priority queue	A data structure that quickly returns the node with the smallest frequency.
Frequency	How many times a character appears in the input.

Huffman algorithm — steps

• Count the frequency of every distinct character.

• Create one leaf node per character and insert all leaves into a min-heap.

• Remove the two nodes with the smallest frequencies.

• Create a parent node whose frequency is the sum of those two frequencies; attach them as children.

• Insert the parent back into the min-heap.

• Repeat until only one node remains; it is the root of the Huffman tree.

• Traverse from root: write 0 when moving left and 1 when moving right. The root-to-leaf path is that character's code.

Pseudocode

```
HUFFMAN(nodes): Q = min_heap(nodes) while size(Q) > 1: x = extract_min(Q) y = extract_min(Q) z = new_node(x.freq + y.freq) z.left = x; z.right = y insert(Q, z) return extract_min(Q)
```

Formulas

```
Combined frequency = frequency(x) + frequency(y) Total encoded bits = SUM(frequency_i * code_length_i) Average code length = Total encoded bits / Total number of characters
```

Example from the supplied PDF: frequencies a=5, b=9, c=12, d=13, e=16, f=45. The first merge is 5+9=14. The next merges repeatedly combine the two smallest remaining frequencies until the root frequency is 100.

Complexity

Operation / method	Time complexity	Reason
Build heap	O(k)	Heap can be built bottom-up.
Huffman tree using min-heap	O(k log k)	There are k-1 merges; each extracts two minima and inserts one combined node.
Print codes by traversing tree	O(k)	Visits the tree's O(k) nodes.
Count frequencies in input of length m	O(m)	Reads each input character once (assuming efficient frequency storage).
Space	O(k)	Tree and priority queue store O(k) nodes.

Here k = number of distinct characters and m = total input length. The assignment PDF states O(n log n) for Huffman construction and notes a linear-time method when the input frequencies are already sorted.

Viva questions

• Why is Huffman coding lossless? Prefix codes make decoding unambiguous, so the original message can be reconstructed.

• Why use a min-heap? It lets us repeatedly select the two least-frequent nodes efficiently.

• Which tree is used? A binary Huffman tree.

• Which character gets the shortest code? Generally, the most frequent character.

4. Practical 3 — 0/1 Knapsack (Dynamic Programming)

Definitions

Term	Easy meaning
Optimization problem	A problem where we want the best value, minimum cost, maximum profit, etc.
Dynamic Programming (DP)	Solves subproblems once, stores their answers and reuses them.
Overlapping subproblems	The same smaller problem appears repeatedly.
Optimal substructure	An optimal overall solution can be built from optimal solutions to smaller subproblems.
0/1 Knapsack	Each item is either selected completely (1) or not selected (0); fractions are not allowed.
Capacity (W)	Maximum weight the bag can carry.
Profit/value	Benefit gained by selecting an item.

DP recurrence / formula

```
DP[i][w] = 0, if i = 0 or w = 0 If weight[i] > w: DP[i][w] = DP[i-1][w] Otherwise: DP[i][w] = max( DP[i-1][w], value[i] + DP[i-1][w - weight[i]] )
```

Meaning: Compare (1) leaving the current item out and (2) taking it plus the best value possible with the remaining capacity. Choose the larger value.

Steps to solve

• Create a table with (n+1) rows and (W+1) columns.

• Fill row 0 and column 0 with zero.

• Fill the table row by row using the recurrence.

• The bottom-right cell gives the maximum achievable value.

• Trace back through the table to identify which items were selected.

Complexity

Part	Time	Space
Fill DP table	O(nW)	O(nW) table
Trace selected items	O(n)	O(1) extra, excluding output
Overall standard DP	O(nW)	O(nW)
Space-optimized DP (one-dimensional)	O(nW)	O(W)

Important: O(nW) is called pseudo-polynomial time because W is a numeric capacity, not the number of bits needed to represent W. The assignment's table method uses O(nW) time and O(nW) space.

DP vs Greedy vs Branch and Bound

Approach	Main idea	For 0/1 Knapsack
Greedy	Select according to a local rule, such as highest value/weight ratio.	Does not guarantee the optimal 0/1 answer in general.
Dynamic Programming	Stores best results for item prefixes and capacities.	Gives an optimal answer; standard complexity O(nW).
Branch and Bound	Explores a state-space tree and prunes branches using bounds.	Can find an optimum; running time can be exponential in the worst case.

Viva points

• Why is it called 0/1? An item is either taken (1) or not taken (0).

• Can we take half an item? No; that is the fractional knapsack problem.

• Why DP? Subproblems overlap and the problem has optimal substructure.

• Is it polynomial? O(nW) is pseudo-polynomial; the problem is NP-complete in its decision form and NP-hard in its optimization form.

5. Practical 4 — N-Queens using Backtracking

Definitions

N-Queens: Place N queens on an N by N chessboard so that no two queens attack each other.

Backtracking: Build a solution step by step. If a choice makes the solution impossible or invalid, undo it and try another choice.

Safety conditions

• No two queens can be in the same row.

• No two queens can be in the same column.

• No two queens can be on the same diagonal.

When placing one queen per row, the row condition is automatically satisfied. The algorithm checks the column and both diagonals against previously placed queens.

Algorithm steps

• Create an N by N board and place the first queen at the required fixed position.

• Move to the next row and try each column.

• Check whether the position is safe: same column, upper-left diagonal and upper-right diagonal.

• If safe, place the queen and recursively solve the next row.

• If no position works, remove the previous queen and try its next position.

• Stop when all N queens are placed, or all choices have been explored.

Pseudocode

```
SOLVE(row): if row == N: return true for col = 0 to N-1: if SAFE(row, col): board[row][col] = 'Q'
if SOLVE(row + 1): return true board[row][col] = '.' // undo choice return false
```

Complexity

Measure	Complexity	Explanation
Time (standard row-by-row search)	O(N!) upper bound	At each row, columns already used cannot be reused; pruning rejects conflicts early. Exact work depends on fixed queen and pruning.
Auxiliary space	O(N)	Recursion depth and column/diagonal tracking.
Board storage	O(N^2)	The chessboard itself has N rows and N columns.

N=4 example from the PDF

```
Q . . . . . Q . . . . Q . Q . .
```

Queen positions: (row 1, col 1), (row 2, col 3), (row 3, col 4), (row 4, col 2). This is a valid solution because no two queens share a column or diagonal.

Applications of backtracking

• Sudoku solving

• Maze solving

• Graph coloring

• Subset-sum problems

6. Practical 5 — Deterministic and Randomized Quick Sort

Definitions

Term	Easy meaning
Quick Sort	A divide-and-conquer sorting algorithm: partition around a pivot, then sort the two sides.
Pivot	The element used to divide the array into smaller and larger elements.
Partition	Rearranges elements so they lie on the appropriate side of the pivot.
Deterministic Quick Sort	Chooses the pivot by a fixed rule, such as first, last or middle element.
Randomized Quick Sort	Chooses the pivot randomly, reducing dependence on the input's original order.
Randomized algorithm	Uses random choices during execution.
Las Vegas algorithm	Always gives a correct result, but its running time can vary randomly.
Monte Carlo algorithm	Has a bounded/deterministic running time but may give an incorrect result with some probability.

Quick Sort procedure

• Choose a pivot.

• Partition the array around the pivot.

• Recursively sort the left part.

• Recursively sort the right part.

• Stop when a part has zero or one element.

Randomized Quick Sort pseudocode

```
RANDOMIZED_QUICKSORT(A, low, high): if low < high: r = RANDOM(low, high) swap A[r] with A[high] p = PARTITION(A, low, high) RANDOMIZED_QUICKSORT(A, low, p-1) RANDOMIZED_QUICKSORT(A, p+1, high)
```

Time and space complexity

Case	Time complexity	When it happens
Best	O(n log n)	Partitions are approximately balanced.
Average / expected	O(n log n)	Expected behavior with varied/random pivots.
Worst	O(n^2)	Pivot repeatedly becomes the smallest or largest element, creating very unbalanced partitions.

Space	Complexity
Average recursion stack	O(log n)
Worst recursion stack	O(n)
Stability	Standard in-place Quick Sort is not stable; swaps may change the relative order of equal elements.

Randomization improves expected performance and reduces dependence on input order, but it does not remove the O(n^2) worst case.

Randomized Quick Sort comparison analysis

```
X = total number of comparisons X = SUM over all pairs (i, j) of I{zi and zj are compared} E[X] = SUM over all pairs P(zi and zj are compared)
```

The assignment uses an indicator random variable: I is 1 if a particular pair of sorted elements is compared, otherwise 0. Expected comparisons lead to expected running time O(n log n).

Quick Sort vs Merge Sort

Quick Sort	Merge Sort
Usually in-place apart from recursion stack.	Usually needs O(n) extra array space.
Average/expected O(n log n); worst O(n^2).	O(n log n) in best, average and worst cases.
Standard version is not stable.	Standard merge sort is stable.
Often good cache behavior and practical speed.	Predictable worst-case performance.

Randomized algorithm applications listed in the PDF

• Cryptography

• Load balancing

• Primality testing

• Hashing, sorting, searching and order statistics

• Computational geometry

• Polynomial and matrix identity verification

• Linear programming and rounding

7. Master complexity comparison — memorize for viva

Algorithm / method	Best	Average / expected	Worst	Auxiliary space
Fibonacci — naive recursion	—	—	O(2^n) upper bound	O(n)
Fibonacci — DP array	O(n)	O(n)	O(n)	O(n)
Fibonacci — optimized iteration	O(n)	O(n)	O(n)	O(1)
Fibonacci — fast doubling	O(log n)	O(log n)	O(log n)	O(log n) recursive
Huffman construction (min-heap)	—	O(k log k)	O(k log k)	O(k)
0/1 Knapsack DP	—	O(nW)	O(nW)	O(nW)
N-Queens backtracking	Depends on pruning	Depends on pruning	O(N!) upper bound	O(N) auxiliary + O(N^2) board
Quick Sort	O(n log n)	O(n log n) expected	O(n^2)	O(log n) avg; O(n) worst

Symbols: n = input length / number of items as appropriate; k = distinct characters; W = knapsack capacity; N = chessboard size. A dash means the supplied practical does not emphasize that case.

8. Short viva questions and answers

1. What is DAA? Design and Analysis of Algorithms studies how algorithms are designed and how their time and memory use are analyzed.

2. What is an algorithm? A finite sequence of clear steps to solve a problem.

3. What is time complexity? How running work grows as input size increases.

4. What is space complexity? How memory usage grows as input size increases.

5. What is recursion? A function calling itself on a smaller problem.

6. What is iteration? Repeating operations using loops.

7. What is the Fibonacci recurrence? F(n) = F(n-1) + F(n-2), with F(0)=0 and F(1)=1.

8. Why is naive Fibonacci recursion slow? It recalculates the same Fibonacci values many times.

9. Time complexity of iterative Fibonacci? O(n); space is O(1) when only two previous values are stored.

10. What is memoization? Storing results of recursive calls so they are not calculated again.

11. What is the golden ratio? Approximately 1.618; ratios of consecutive Fibonacci numbers approach it.

12. What is a greedy algorithm? It chooses the locally best available option at each step.

13. What is Huffman coding? A lossless compression algorithm that assigns variable-length prefix codes based on character frequency.

14. Why is Huffman coding lossless? Prefix codes allow the encoded message to be decoded unambiguously.

15. Which data structure is used in Huffman coding? A min-heap / priority queue.

16. Why do we extract two minimum nodes? Huffman repeatedly combines the two least-frequent nodes to build an optimal prefix-code tree.

17. Huffman construction complexity? O(k log k) using a min-heap, where k is the number of distinct symbols.

18. What is a prefix code? No complete codeword is the prefix of another codeword.

19. What is Dynamic Programming? Solving subproblems once and storing their answers for reuse.

20. What are overlapping subproblems? The same smaller problem is needed more than once.

21. What is optimal substructure? An optimal solution can be built from optimal solutions to smaller subproblems.

22. Explain 0/1 Knapsack. Each item must be either fully selected or completely rejected.

23. What is the time complexity of 0/1 Knapsack DP? O(nW), where n is the number of items and W is capacity.

24. Why is O(nW) pseudo-polynomial? It depends on the numeric value W, not on the number of bits used to encode W.

25. What is backtracking? Trying a choice, checking it, and undoing it if it cannot lead to a valid solution.

26. What is the N-Queens condition? No two queens can share a row, column or diagonal.

27. N-Queens complexity? O(N!) upper bound for the standard row-by-row search; auxiliary space O(N), plus O(N^2) board storage.

28. What is a pivot in Quick Sort? The element used to partition the array.

29. What is deterministic Quick Sort? Quick Sort that selects its pivot using a fixed rule.

30. What is randomized Quick Sort? Quick Sort that selects its pivot randomly.

31. Quick Sort worst-case time? O(n^2), when partitions are repeatedly highly unbalanced.

32. Randomized Quick Sort expected time? O(n log n).

33. Is Quick Sort stable? Standard in-place Quick Sort is not stable.

34. What is a Las Vegas algorithm? It always returns a correct result, but its running time may be random.

35. What is a Monte Carlo algorithm? It has a bounded running time but can return an incorrect result with some probability.

36. Quick Sort vs Merge Sort? Quick Sort is often in-place and fast on average; Merge Sort guarantees O(n log n) worst-case time and is stable, but usually uses O(n) extra memory.

9. Formula sheet — last-minute revision

Topic	Formula / key fact
Fibonacci	F(0)=0; F(1)=1; F(n)=F(n-1)+F(n-2)
Golden ratio	Approximately 1.618
Huffman merge	New node frequency = frequency 1 + frequency 2
Huffman encoded size	Total bits = SUM(frequency_i * code_length_i)
Huffman average code length	Total encoded bits / total number of characters
0/1 Knapsack	DP[i][w] = max(DP[i-1][w], value[i] + DP[i-1][w-weight[i]]) if item fits; otherwise DP[i-1][w]
Quick Sort recurrence (balanced)	T(n) = 2T(n/2) + O(n) = O(n log n)
Quick Sort recurrence (unbalanced)	T(n) = T(n-1) + O(n) = O(n^2)
Randomized expected time	O(n log n)
N-Queens safety	Check column and both diagonals for each new row.

Source coverage and notes: Prepared from the five uploaded PDFs: Assignment 1 Fibonacci (6 pages), Assignment 2 Huffman Encoding (5 pages), Assignment 3 0/1 Knapsack (6 pages), the uploaded N-Queens file (2 pages; its title page labels it Assignment No. 5), and Quick Sort variants (4 pages). The PDFs ask additional viva/assignment questions; this guide includes concise answers for the main concepts. Complexity labels are presented in standard form where the source gives incomplete detail; the distinction between the source's stated result and the standard analysis is explained in the relevant sections.